Software Engineering Project May-2026

Milestone 5 Deliverables for ZenStudy

Submitted By

Team Name: **DeerInTheCode**

Team Code : **Team-051**



IITM Online BS Degree Program,
Indian Institute of Technology, Madras,
Chennai Tamil Nadu, India, 600036

Community Services Platform for Library Management

Project Details

Software Name: ZenStudy

Department Context: Library Management System

System Type: Small Business Logistics & Community Services Platform

Date of Submission: 23-08-2026

Team Details

Team Name: DeerInTheCode
Team Code: Team-051

Name	Roll Number	Role
DIKSHA SHARMA	23f2004899	Product Manager
KARTHIK V N	23f1000033	Frontend Developer
NEHA CHACKO	24f1002416	Scrum Master
GAURAV TOMAR	23f1000805	Backend Developer
AKSHATHA A	23f1001878	Code Reviewer/Tester

Table of Contents

1. Executive Summary
2. Project Background
3. Problem Statement
4. Identification of Users
5. User Research and Pain Point Analysis
6. User Stories
7. System Design and Architecture
8. Sprint Schedule
9. Gantt Chart
10. Kanban Board
11. Tools and Technologies Used
12. Implementation Details
13. Testing, Quality Assurance and User Feedback
14. Issue Tracking, Code Review and Project Management
15. Instructions to Run the Application
16. Challenges Faced and Lessons Learned
17. Conclusion and Future Enhancements

Executive Summary

This project delivers a Study Library Management System that digitizes day-to-day library operations for administrators, staff, and students. Built with a Flask REST API, SQLite, and Vue.js dashboards, it replaces fragmented manual processes with a single, role-based web application.

The system covers the full membership and seating lifecycle: student registration and admin approval, membership plans and booking rules, seat reservations with check-in / check-out and no-show handling, walk-in bookings, extensions, offline payment recording, complaints, and announcements. Administrators manage staff accounts, operating hours, seat capacity and zones, and view live occupancy and revenue analytics. Staff handle floor operations; students book seats and manage their own profiles, bookings, payments, and complaints, subjected to active membership.

Key outcomes include secure JWT-based access control, enforcement of booking and membership rules, and real-time reporting grounded in live database data rather than static demo figures. Automated tests support core API behavior. Overall, the solution improves operational visibility, reduces seat conflicts and no-shows, and provides a clear, scalable foundation for library resource management.

Project Background

Study libraries and shared reading spaces face growing demand from students who need quiet, bookable seats for focused work. In many institutions these spaces are still managed with paper registers, ad-hoc phone calls, or informal verbal allotments. That approach does not scale: seats get double-booked, occupancy is hard to track, no- shows leave desks unused, membership and fee records stay scattered, and administrators lack a clear picture of usage and revenue.

Staff spend time on routine tasks such as checking who is allowed to enter, finding free seats for walk-ins, recording payments, and handling complaints, while students lack a reliable way to reserve space in advance or see the status of their membership and bookings. Without defined rules (advance booking limits, slot duration, cancellation windows, check-in deadlines), fairness and utilization both suffer.
Against this backdrop, the present project develops a Study Library Management System for coordinated, role-based digital operations.

The system is intended to support three primary stakeholders:

- Administrators, who configure plans, hours, capacity, staff accounts, and analytics
- Library staff, who manage daily floor operations (check-in, walk-ins, extensions, payments, complaints)
- Students, who register for membership, book seats, and track their own history

By moving these workflows onto a unified web platform with a backend API and dedicated dashboards, the project aims to improve seat utilization, reduce administrative overhead, enforce consistent membership and booking policies, and provide live operational insight, addressing the limitations of traditional, manual library seat management.

Problem Statement

Study libraries and reading rooms are increasingly popular among students seeking quiet and dedicated spaces for learning. However, many study libraries continue to rely on manual or semi-digital processes to manage memberships, seat bookings, attendance, payments, and daily operations. These traditional methods often lead to inefficiencies, operational delays, and poor resource utilization.

Students frequently face challenges such as the lack of real-time information on seat availability, difficulty in reserving study spaces, limited visibility into their bookings and membership status, and inconvenience when requesting booking extensions or resolving issues. Manual check-in procedures and the absence of a centralized platform also result in longer waiting times and reduced user satisfaction.

From an operational perspective, library staff must manually manage walk-in admissions, verify bookings, process offline payments, allocate seats, track attendance, handle no-shows, and resolve complaints. These repetitive administrative tasks increase the likelihood of human error and make it difficult to efficiently utilize available study spaces. Additionally, library administrators often lack comprehensive insights into occupancy trends, peak usage hours, membership statistics, and revenue, limiting their ability to make informed decisions regarding resource allocation and policy improvements.

Therefore, there is a need for a centralized Study Library Management System that streamlines membership management, seat booking, attendance tracking, payment recording, and administrative operations while providing real-time information to students, supporting efficient daily operations for staff, and enabling data-driven decision-making

for administrators. Such a system would improve operational efficiency, optimize space utilization, and enhance the overall study library experience for all stakeholders.

Identification of Users

1. Primary Users:

These are the core operational users of the Study Library Management System. The platform's daily workflows, including membership verification, seat reservations, check-ins, payments and occupancy management, are primarily designed around their interactions. Their continuous use ensures the smooth functioning of the study library.

1.1 Library Staff: Library Staff are responsible for the day-to-day operations of the study library. Their responsibilities include:

- Registering new students and verifying memberships.
- Verifying student bookings and performing check-ins using the student's unique check-in code.
- Recording payments (cash, UPI, card, etc.).
- Marking no-shows and releasing unoccupied reserved seats.
- Handling student complaints and providing operational support.

The system must provide an efficient operational dashboard that enables staff to manage bookings, occupancy, payments, and student services with minimal manual effort.

1.2 Student: Students are the primary beneficiaries of the study library services and interact with the platform to access study facilities. Their responsibilities include:

- Logging in to access their account
- Viewing membership details and validity.
- Checking real-time seat and study room availability and payment history
- Booking study seats or rooms in advance.
- Modifying or cancelling bookings within the permitted period.
- Presenting their unique check-in code to library staff upon arrival.
- Checking out after completing a study session.
- Raising complaints or support requests when necessary.

The system must provide students with a simple and convenient platform for reserving study spaces, tracking their activity, and communicating with library staff.

2. Secondary Users:

These users oversee the overall functioning of the study library and are responsible for administrative decisions rather than daily operational activities.

Admin: Admin supervises the entire study library management system and ensures that policies and configurations are maintained effectively. Responsibilities include:

- Managing library staff accounts.
- Approving and managing student memberships.
- Configuring operating hours, booking limits and membership plans.
- Viewing reports and statistics related to occupancy, peak hours, memberships and revenue.
- Managing announcements and notifications for users.

The system must provide comprehensive administrative controls, configurable system settings, and analytical dashboards to support informed decision-making and efficient management.

3. Tertiary Users:

The proposed Study Library Management System does not include any tertiary users. All identified stakeholders who directly interact with the platform are classified as either primary or secondary users based on their operational responsibilities and level of system interaction.

User Research & Pain Point Analysis

The objective of the user research was to understand the operational challenges faced by study libraries and identify the needs of different stakeholders, including administrators, library staff, and students. The research focused on existing workflows related to membership management, seat booking, attendance tracking, payments, and communication to determine opportunities for improving efficiency through a Study Library Management System (SLMS).

Key Observational Insights

The following observations were made after studying the existing workflow and interacting with stakeholders:

- Most study libraries rely on manual registers or spreadsheets to manage student memberships, attendance, and seat allocation.
- Staff spend significant time verifying memberships, assigning seats, and recording daily attendance.
- Students often visit the library without knowing whether seats are available, leading to unnecessary waiting.
- Booking modifications and cancellations are managed manually, making it difficult to utilize vacant seats effectively.
- Payment records are maintained separately, increasing the risk of errors and making transaction tracking difficult.
- Communication regarding announcements, membership expiry, and complaints is handled informally through phone calls or messaging applications.
- Administrators lack centralized reports on occupancy, memberships, revenue, and peak usage hours, making operational planning challenging.

Stakeholder Pain Points

1.1 Students

- Difficulty in checking seat availability before visiting the library.
- No facility to reserve study seats in advance.
- Lack of visibility into membership validity and renewal dates.
- Difficulty in accessing payment history and booking records.
- Limited communication regarding announcements, booking confirmations, and complaint resolution.
- Time wasted waiting for seat allocation during busy hours.

Core Issue: Students face difficulty accessing study library services efficiently due to limited visibility into seat availability, memberships, and bookings.

1.2 Library Staff

- Manual student registration and membership verification consume time.
- Maintaining attendance registers is repetitive and prone to human error.
- Managing walk-in students alongside advance bookings is difficult.
- Tracking payments manually increases administrative workload.
- Handling complaints without a centralized tracking system delays resolution.
- Difficulty identifying no-show students and reallocating unused seats promptly.

Core Issue: Library staff spend excessive time performing repetitive manual tasks, reducing operational efficiency and increasing the likelihood of errors.

1.3 Admin

- Limited visibility into overall library performance.
- Difficulty monitoring occupancy trends and peak usage hours.
- Time-consuming management of memberships and membership plans.
- Lack of consolidated reports for revenue, attendance, and memberships.
- Inefficient communication with students regarding important announcements.
- Limited ability to configure and enforce booking policies consistently.

Core Issue: Administrators lack centralized control and real-time operational insights, making effective management and decision-making challenging.

Root Causes Identified

The primary causes contributing to the identified challenges include:

- Absence of a centralized digital platform for managing study library operations.
- Manual processes for bookings, attendance, payments, and memberships.
- Lack of real-time synchronization of seat availability.
- Limited automation for routine administrative tasks.
- Fragmented storage of operational data across multiple records.
- No integrated notification mechanism for students and staff.
- Limited reporting and analytics capabilities for decision-making.

Overall Conclusion

The user research indicates that the current management of study libraries is largely manual, resulting in inefficiencies, inconsistent records, and increased administrative effort. Students experience inconvenience due to limited visibility into seat availability, memberships, and bookings, while library staff spend considerable time on repetitive operational tasks. Administrators lack comprehensive insights needed for effective decision-making.

The proposed **Study Library Management System (SLMS)** addresses these challenges by providing centralized membership management, real-time seat booking, attendance tracking, payment management, complaint handling, automated notifications, and analytical reporting. By digitizing these processes, the system aims to improve operational efficiency, reduce manual errors, enhance resource utilization, and deliver a better overall experience for administrators, staff, and students.

User Stories

Epic 1: User & Membership Management

Story 1.1: Staff Account Management

- **As an** Admin,
- **I want** to create, update, and deactivate library staff accounts,
- **So that** only authorized staff members can access the system.

Story 1.2: Student Membership Management

- **As an** Admin,
- **I want** to approve and manage student memberships,
- **So that** only eligible students can use the study library.

Story 1.3: Student Registration

- **As a** Library Staff,
- I **want** to register new students and verify their memberships
- **So that** students can access the study library without delays.

Story 1.4: Student Account & Membership Information

- **As a** student,
- I **want** to securely log in to my account, reset my password if needed, and view my membership details and validity,
- **So that** I can securely access the system, manage my account, and know my membership status and renewal date

Story 1.5: Profile Management

- **As a** Student,
- **I want** to update my personal details,
- **So that** my contact information remains up-to-date

Epic 2: System Administration & Configuration

Story 2.1: Operating Hours Configuration

- **As an** Admin,
- **I want** to configure the operating hours of the study library
- **So that** students can only book and access the library during working hours.

Story 2.2: Booking Limit Configuration

- **As an** Admin,
- **I want** to set booking limits, including the number of available seats,
- **So tha**t occupancy is managed efficiently.

Story 2.3: Membership Plan Management

- **As an** Admin ,
- **I want** to create and manage membership plans,
- **So that** students can subscribe to the plan that best suits their needs.

Story 2.4: Library Reports & Analytics

- **As an** Admin ,
- **I want** to view reports and statistics on occupancy, peak hours, memberships, and revenue,
- **So that** I can make informed operational decisions.

Epic 3: Booking and Seat Management

Story 3.1: Real-Time Seat Availability

- **As a** Student,
- **I want** to view real-time seat and study room availability
- **So that** I can choose an available space.

Story 3.2: Advance Booking

- **As a** Student,
- **I want** to book study seats or rooms in advance
- **So that** I can reserve a study space before arriving

Story 3.3: Booking Modification

- **As a** Student,
- **I want** to modify or cancel my booking within the permitted time
- **So that** I can adjust my reservation if my plans change

Story 3.4: Booking and Attendance History

- **As a** Student,
- **I want** to view my booking and attendance history
- **So that** I can review my study sessions

Story 3.5: Daily Booking Overview

- **As a** Library Staff,
- **I want** to view daily bookings and expected arrivals
- **So that** I can prepare for incoming students

Story 3.6: Walk-In Admission

- **As a** Library Staff,
- **I want** handle same-day walk-in admissions by assigning available seats
- **So that** students without reservations can use the facility

Story 3.7: Extension Approval & Seat Allocation

- **As a** Library Staff,
- **I want** to review extension requests and either extend the current booking or assign a new available seat,
- **So that** students can continue their study sessions while ensuring efficient seat utilization.

Epic 4: Check-in & Attendance Management

Story 4.1: Student Check-In

- **As a** student,
- **I want** to present my unique check-in code to the library staff,
- **So that** my attendance can be verified before I access the study library.

Story 4.2: Check-In Verification

- **As a** Library Staff,
- **I want** to verify booking details using the student's check-in code,
- **So that** only students with valid bookings are checked in.

Story 4.3: No-Show Management

- **As a** Library Staff,
- **I want** to mark no-show bookings after waiting time and release reserved seats,
- **So that** unused seats become available to other students

Story 4.4: Study Session Check-Out

- **As a** Student,
- **I want** to check out after completing my study session,
- **So that** my attendance is recorded accurately and my seat becomes available for other students.

Epic 5: Payment Management

Story 5.1: Offline Payment Recording

- **As a** Library Staff,
- **I want** to record student payments and specify the payment mode, such as cash, UPI, or card,
- **So that** all transactions are documented accurately.

Story 5.2: Payment History

- **As a** Student,
- **I want** to view my payment history,
- **So that** I can keep track of all my library transactions.

Epic 6: Complaints & Communication

Story 6.1: Complaint Submission

- **As a** Student,
- **I want t**o raise complaints to the library staff,
- **So that** my issues can be addressed promptly.

Story 6.2: Complaint Management

- **As a** Library Staff,
- **I want** to view and resolve student complaints,
- **So that** reported issues are handled efficiently and in a timely manner.

Story 6.3: Announcements & Notifications

- **As an** Admin,
- **I want** to create and manage announcements and notifications
- **So that** important information can be communicated to all users.

System Design and Architecture

Class Diagram

The UML Class Diagram represents the structure of the Study Library Management System (SLMS) by illustrating the key classes, their attributes, methods, and relationships. It models the interactions between Students, Library Staff, and Admin, along with core entities such as Membership, Booking, Seat, Attendance, Payment, Complaint, Announcement, and Report, providing a clear blueprint for the system's design.

Link for class diagram: Class diagram

ER Diagram

The Entity Relationship (ER) diagram illustrates the database structure of the Study Library Management System by defining the key entities, their attributes, and the relationships between them. It models the core functionalities of the system, including user management, memberships, seat bookings, attendance tracking, payments, complaints, announcements, and library settings. The database is designed using a relational model to ensure data consistency, minimize redundancy, and efficiently support the day-to-day operations of the study library.

Link for ER Diagram: ER Diagram

Sprint Schedule

Sprint 1: Project Initiation & Planning

- **Duration: 18/06/2026 – 21/06/2026**
- Finalize project topic and scope
- Identify primary stakeholders
- Finalize user research strategy
- Prepare interview questionnaire and observation plan
- Assign team roles and responsibilities
- Plan project milestones and timeline

Sprint 2: User Research & Requirements Engineering

- **Duration: 22/06/2026 – 28/06/2026**
- Conduct stakeholder interviews and observations
- Analyze interview responses and identify pain points
- Identify functional and non-functional requirements
- Define system features
- Create product backlog
- Write SMART user stories and epics
- Validate requirements with stakeholders
- Prepare Milestone 1 deliverables (Problem Statement, Users, Pain Points, User Stories)

Sprint 3: System Planning & Design

- **Duration: 29/06/2026 – 05/07/2026**
- Finalize project branding and technology stack
- Create project schedule and Gantt chart
- Set up Jira project and task assignments
- Design system architecture
- Create Component Diagram
- Create UML Class Diagram
- Design database schema (ER Diagram)
- Develop UI wireframes and page navigation
- Prepare design documentation

Sprint 4: Frontend Development (Phase I)

- **Duration: 06/07/2026 – 14/07/2026**
- Develop authentication and user management pages
- Develop membership management pages
- Develop admin dashboard and configuration pages
- Develop responsive UI components
- Perform frontend unit testing

Sprint 5: Frontend Development (Phase II)

- **Duration: 15/07/2026 – 23/07/2026**
- Develop booking and seat management pages

- Develop check-in and attendance pages
- Develop payment management pages
- Develop complaints and communication pages
- Integrate all frontend pages
- **Complete all frontend screens by 23/07/2026**

Sprint 6: Backend Development - 1

- **Duration: 24/07/2026 – 03/08/2026**
- Develop backend APIs of first 3 epics
- Implement business logic and database integration
- Implement authentication and role-based access
- Perform API testing

Sprint 7: Backend Development – 2

- **Duration: 04/08/2026 – 12/08/2026**
- Develop remaining backend APIs
- Implement business logic and database integration
- Perform API testing

Sprint 8: Integration

- **Duration: 13/08/2026 – 18/08/2026**
- Integrate frontend with backend APIs
- Perform API testing and fix integration issues

Sprint 9: Testing & Quality Assurance

- **Duration: 18/08/2026 – 20/08/2026**
- Execute unit, integration, and system testing
- Perform bug fixing and regression testing
- Validate user stories and acceptance criteria
- Conduct User Acceptance Testing (UAT)

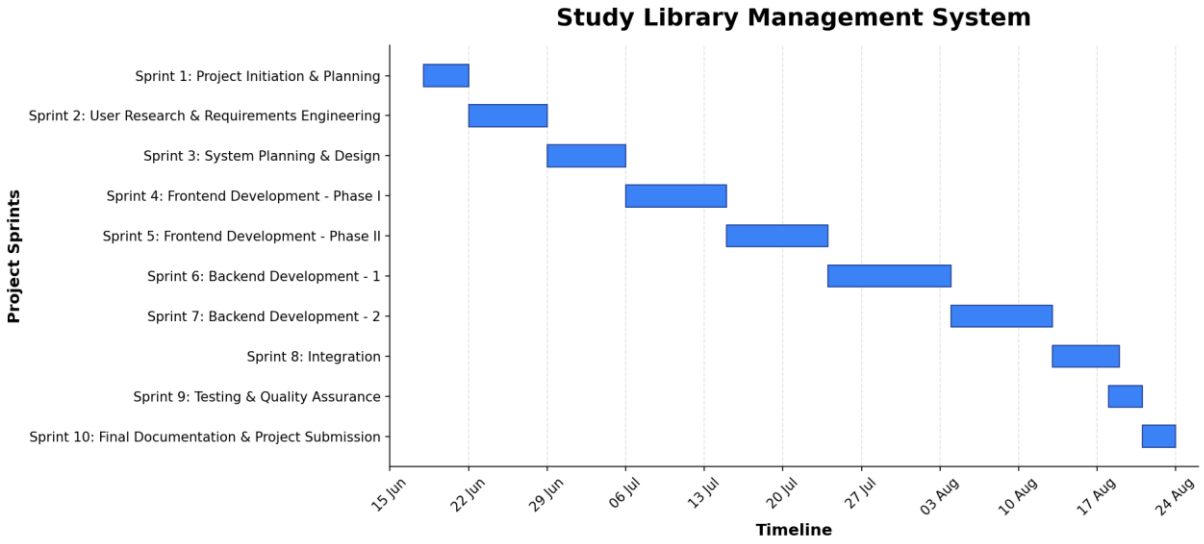
Sprint 10: Final Documentation & Project Submission

- **Duration: 21/08/2026 – 23/08/2026**
- Finalize project documentation
- Update README and deployment guide
- Prepare final project report and presentation

- Perform final system validation
- Deploy the application
- Submit all project deliverables

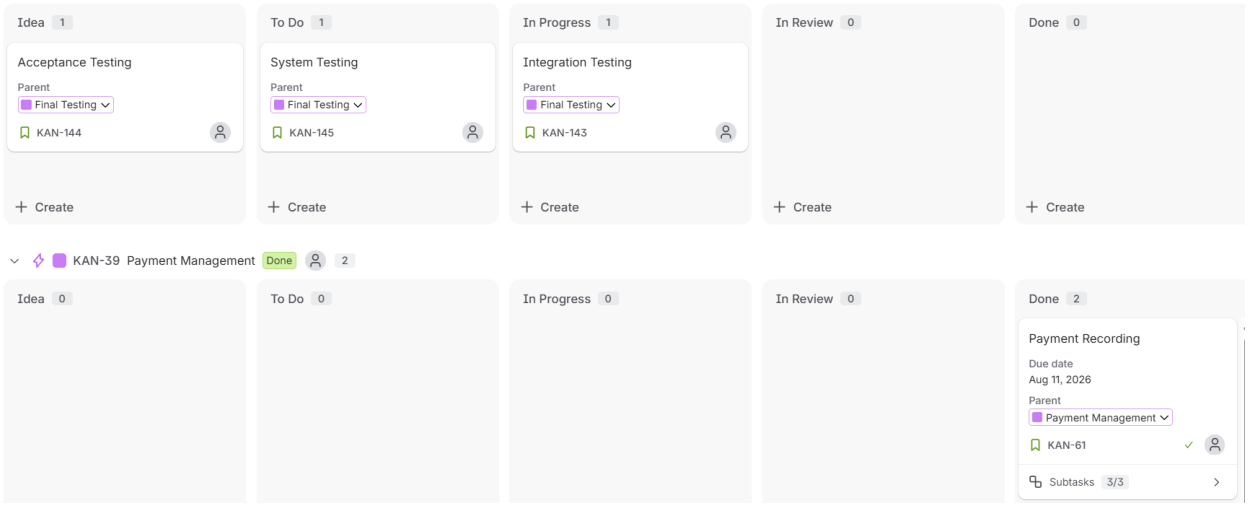
Gantt Chart

The Gantt chart illustrates the planned timeline for the Study Library Management System project, spanning from 18th June to 23rd August 2026. It divides the project into eight sequential sprints covering project planning, requirements gathering, system design, frontend development, backend integration, testing, and final documentation. A milestone is included to indicate the Frontend Freeze on 23 July, ensuring all user interface development is completed before integration and testing. The schedule provides a structured roadmap for managing project activities and tracking progress throughout the software development lifecycle.



Kanban Board

The Kanban board provides a visual representation of the project workflow by organizing tasks into different stages based on their current status, such as To Do, In Progress, and Done. It groups related tasks under their respective epics, enabling the team to monitor progress, prioritize work, assign responsibilities, and track the completion of project activities. This workflow-based approach improves collaboration, enhances transparency, and ensures efficient management of the Study Library Management System development process.



Tools and Technologies Used

1. **Backend**
   - Python 3
   - Flask
   - Flask-RESTful
   - Flask-SQLAlchemy
   - Flask-JWT-Extended
   - Flask-CORS
   - Werkzeug (password hashing)
   - SQLAlchemy

2. **Database:** SQLite (app.db)
3. **Frontend**
   - HTML5
   - CSS3
   - JavaScript (ES6+)
   - Vue.js 3 (CDN / global build)
   - Jinja2 templates (Flask)

4. **API & Security**
   - REST API (/api/...)
   - JWT authentication
   - Role-based access (Admin / Staff / Student)

5. **Testing**
    - pytest
    - Flask test client
6. **Development & Version Control**
    - GitHub
    - VS Code for IDE
    - Windows PowerShell

Architecture style

- Client–server web app (Flask serves UI + API)
- MVC-style separation (models, resources/controllers, templates/static views

Implementation Details

The application follows a client–server design. A single Flask app serves both the REST API and the web UI (Jinja templates + static JS/CSS). Role-specific dashboards (Admin, Staff, Student) call the API using JWT tokens stored in the browser. Persistent data is stored in SQLite via SQLAlchemy models.

Backend structure

- app.py: application factory, route registration, SQLite schema upgrades, and seed data (default admin, plans, seats, settings).
- models.py: domain entities: users, students, membership plans, seats, bookings, attendance, payments, complaints, announcements, library settings, extension requests.
- resources/: Flask-RESTful endpoints grouped by feature (auth, staff, memberships, bookings, check-in, no-show, payments, complaints, announcements, reports, settings, profile).
- utils.py: role decorators such as admin_required, staff_or_admin_required, student_required
- config.py: database URI, JWT secrets, token expiry.

Authentication and authorization

Users log in via POST /api/auth/login. Passwords are hashed with Werkzeug. Successful login returns a JWT with role claims. Students may log in only when membership status

is Active; Pending / Rejected / Disapproved accounts are blocked. Protected routes validate the token and enforce role checks.

Core feature implementation

- Memberships: Staff register students; admin approves/rejects/suspends; plans define price and duration; expiry is set on approval.
- Bookings: Students book seats for allowed dates/slots; rules from library settings (duration, advance days, per-day limits, operating hours, holidays) are enforced server-side. Staff support walk-ins and daily booking overview.
- Check-in / check-out / no-show: Staff verify check-in codes; overdue Confirmed bookings appear as no-show candidates after a grace period; marking no-show frees the seat.
- Extensions: Students request longer end times; staff approve (same seat or reassignment) or reject, subject to extension limits.
- Payments: Staff record offline Cash/UPI/Card payments; students and staff can view payment history.
- Complaints & announcements: Students submit complaints; staff update status; admin publishes announcements by audience/priority.
- Admin configuration: Operating hours, holiday closures, booking limits, seat inventory, seat zones, membership plans, and staff accounts.
- Reports: Live occupancy and revenue aggregates from bookings/payments; peak-hours and membership-status statistics for admin analytics.

Frontend implementation: Dashboards are Vue 3 apps embedded in Flask templates (dashboard.html, staff_dashboard.html, admin_dashboard.html). Shared helpers live in library-data.js (time slots, date utilities, badge labels). api.js wraps authenticated fetch calls. UI state is loaded from the API on mount and refreshed for live views (e.g., occupancy/revenue polling).

Data and persistence: SQLite is used for development and demonstration. On startup, db.create_all() creates tables; upgrade_sqlite_schema() adds missing columns for older local databases. Seed data ensures a usable default admin (admin / test-password), sample membership plans, seats, and library hours.

Testing: API behavior is covered with pytest and Flask's test client (test_app.py), including auth, memberships, bookings, check-in, no-show, payments, complaints, announcements, and reports. Fixtures register and approve students before login so tests align with Active-membership login rules.

Testing, Quality Assurance and User Feedback

Testing approach: Quality assurance focused on verifying that API behavior matches the business rules for Admin, Staff, and Student roles. Automated tests were written with pytest and Flask's test client, exercising real request–response flows against an in-memory SQLite database so each run starts from a clean, seeded state.

Automated testing: The suite in test_app.py covers major modules, including:

- Authentication and role-based access (login success/failure, student login only when membership is Active)
- Staff and membership management (registration, approval, rejection, plans)
- Bookings, cancellations, extensions, walk-ins, and seat availability rules
- Check-in, check-out, and no-show candidate / mark-no-show flows
- Offline payments and payment history
- Complaints and announcements
- Admin reports (occupancy, revenue, peak hours, membership stats)

Fixtures create authenticated Admin/Staff/Student users and sample bookings so tests stay independent and repeatable. Failures found during runs (e.g., login before membership approval, past-date booking fixtures, announcement response shape) were corrected so tests align with actual API contracts.

Manual / functional QA

In parallel with automated tests, features were checked through the three dashboards:

- End-to-end flows such as register → approve → login → book → check-in
- Admin settings and reports after save/refresh (profile, hours, limits, live occupancy/revenue)
- Staff floor operations (daily bookings, walk-in, payments, complaints)
- UI correctness (membership distribution colors, upcoming vs history bookings)

Issues found in the UI (hardcoded report data, settings not persisting, "Unassigned" plan labels) were fixed and re-verified by hard-refresh and role-based walkthroughs.

Quality practices

- Role isolation: endpoints return 401/403 when the wrong role calls them
- Server-side rule enforcement: booking limits, operating hours, and membership status are validated in the API, not only in the UI

- Regression awareness: -x / targeted pytest runs were used while fixing fixtures so one failure did not hide later issues
- Data consistency: live reports and seat/zone displays read from the database rather than static demo datasets where features were completed

User feedback: Feedback was gathered informally during development and demo walkthroughs with team members acting as Admin, Staff, and Student users. Common points included:

- Preference for live occupancy and revenue instead of placeholder charts
- Confusion when past bookings still showed as Upcoming
- Expectation that Settings changes survive page refresh
- Clarity needs around membership distribution (Unassigned vs named plans) and distinct plan colors

These comments directly drove UI/API refinements (live reports, profile persistence, booking status by end time, plan names and chart colors). Further formal usability testing with real library staff/students would be a natural next step for production hardening.

Issue Tracking, Code Review and Project Management

Project organization: The work was organized as a team (MAY2026-Team-051) and delivered story by story across epics (authentication and memberships, bookings and seat rules, check-in/no-show, payments, complaints/announcements, admin reports and settings). Each story was implemented, verified against acceptance behavior, then integrated into the shared codebase.

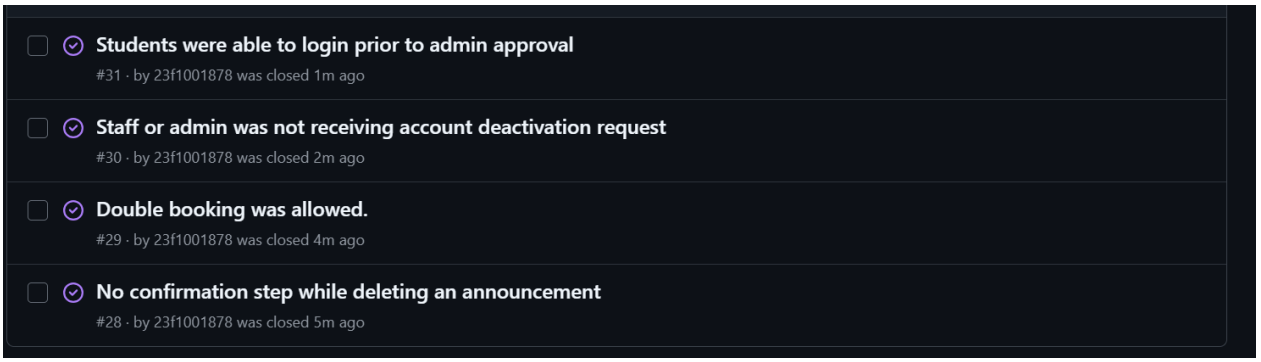
Version control and branching: Git and GitHub were used for source control. Feature work was carried on dedicated branches aligned to stories and epics (for example Story_1.x, Story_2.x, Story_3.x, Epic_6, dev, main). This kept incomplete features isolated, simplified review of related commits, and supported merging into a shared integration line when a story was ready.

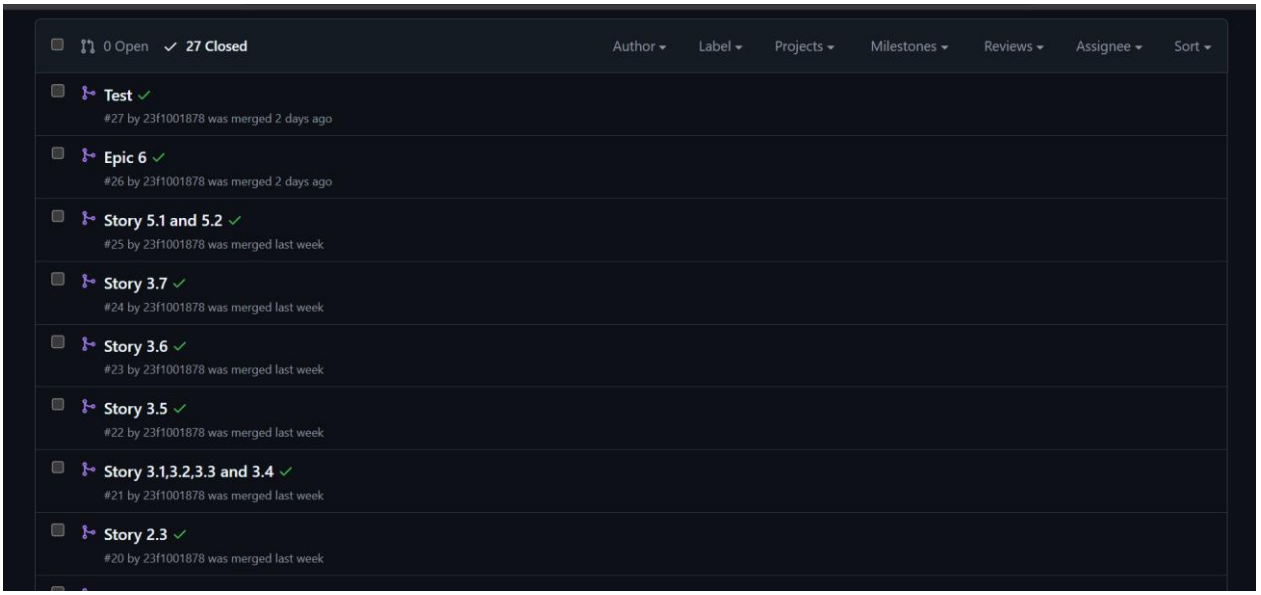
Issue tracking: Issues and tasks were tracked against the product backlog / user stories rather than ad-hoc changes. Typical issue types included:

- New feature stories (e.g., walk-in booking, payment history, live occupancy)
- Defects found in testing or demos (e.g., settings not persisting, upcoming bookings after end time, report 404s, test fixture order)

- Integration fixes (API response shape vs UI expectations, plan names in membership distribution)

Defects discovered during pytest runs or dashboard walkthroughs were noted, fixed on the relevant story branch, and re-tested before considering the story complete.





Code review: Changes were reviewed before or during merge by checking:

- Correct role guards on endpoints
- Consistency between frontend calls and backend contracts
- Avoidance of hardcoded “demo” data where live APIs existed
- Test coverage or fixture updates for new rules (e.g., Active membership required for student login)

Peer / mentor feedback and iterative agent-assisted review in the IDE were used to catch regressions early (wrong login order in fixtures, unregistered report routes, UI mapping bugs).

Project management practices

- **Incremental delivery**: complete one story's backend + UI wiring, then move to the next
- **Definition of done**: API works, dashboard reflects real data where required, and related tests pass
- **Communication of blockers**: time-sensitive tests (walk-in near midnight), fixture vs business-rule mismatches, and schema/API gaps were identified and resolved explicitly

Outcome: This combination of story-based branching, issue-driven fixes, and review-before-integration kept the repository mergeable, reduced last-minute integration risk, and produced a traceable path from requirements → implementation → test → demo for each epic.

Instructions to Run the Application

1. **Prerequisites**

   - Python 3.10+ (project was run with Python 3.13)
   - pip available on PATH
   - Git (to clone the repository)

2. **Clone repository**

   git clone https://github.com/GyaanFlow/MAY2026-Team-051.git

   cd MAY2026-Team-051

3. **Create and activate a virtual environment (recommended)**

   python -m venv venv

   .\venv\Scripts\Activate.ps1

4. **Install dependencies**

   python -m pip install --upgrade pip

   pip install -r requirements.txt

5. **Start the server From the project root:**

   python app.py

Flask starts in debug mode (default: http://127.0.0.1:5000).

Open the application in a browser

Default admin login

- Username: admin
- Password: test-password

Students can log in only after staff registration and admin membership approval (status must be Active). Staff accounts are created by the admin from the Staff Accounts screen.

6. **Run automated tests (optional)**

   python -m pytest test_app.py -q

Challenges faced

- **API and UI mismatches**: The frontend sometimes expected a different response format than the backend returned (for example announcements as a list, nested membership plan details, or specific report field names). This led to screens that looked fine until tests or refresh exposed the gap.
- **Business rules breaking older tests**: Rules such as "students can log in only when membership is Active" and "cannot book a past date" were correct for the product but broke fixtures that logged in too early or created yesterday's bookings through the API. Test helpers had to be redesigned (approve then login; create today then backdate in the database).
- **Time-dependent features**: Walk-in booking, no-show grace periods, and "upcoming vs history" status depend on the current clock. The same test or demo could pass at noon and fail late at night (e.g., walk-in crossing midnight).
- **Cross-role integration**: End-to-end flows span Admin, Staff, and Student. Issues often showed up only at boundaries—pending students unable to log in, plans showing as "Unassigned," or reports returning 404 because routes were not registered.

Lessons learned

- Keep frontend and backend contracts aligned and verify them with automated tests.
- Prefer live API data for anything the user expects to persist or see in reports.
- Update test fixtures whenever product rules change; one shared helper prevents repeated failures.

- Design time-sensitive tests carefully so they do not depend on a lucky wall-clock time.
- Deliver and verify work story by story across all roles before moving on, to catch integration bugs early.

Conclusion

The Study Library Management System successfully digitizes core library operations for administrators, staff, and students. The team delivered authentication and role-based access, membership lifecycle management, seat booking with policy enforcement, check-in/no-show handling, payments, complaints, announcements, configurable library settings, and live admin analytics. Combining a Flask REST backend, SQLite persistence, and Vue.js dashboards produced a coherent, demonstrable product suitable for coursework evaluation and as a foundation for real deployment. Challenges around data contracts, live vs demo UI, and time-sensitive rules strengthened the team's practices in testing, integration, and incremental delivery.

Future enhancements

- Formal schema migrations (e.g., Alembic) instead of ad-hoc SQLite ALTER TABLE upgrades.
- PostgreSQL / hosted database and environment-based configuration for multi-user production use.
- Email/SMS notifications for booking confirmations, approval status, and no-show warnings.
- Online payments and invoices, beyond offline Cash/UPI/Card recording.
- CI pipeline on GitHub Actions to run pytest on every pull request.
- Accessibility and mobile polish for student booking on smaller screens.
- Structured usability testing with real library staff and students to refine workflows.